Aim:

Use Mongoose to connect database for performing CRUD operations

Objective:

- To understand the integration of MongoDB with Node.js using Mongoose.
- To learn how to design and define schemas and models in Mongoose.
- To implement CRUD operations (Create, Read, Update, Delete) on MongoDB collections.
- To practice database connectivity and error handling in Node.js applications.

Technologies Used

- Node.js: JavaScript runtime environment for server-side development.
- MongoDB – NoSQL database for storing and managing data.
- Mongoose – ODM (Object Data Modeling) library for MongoDB and Node.js to simplify database operations.

Step-by-Step Implementation

App.js Code:

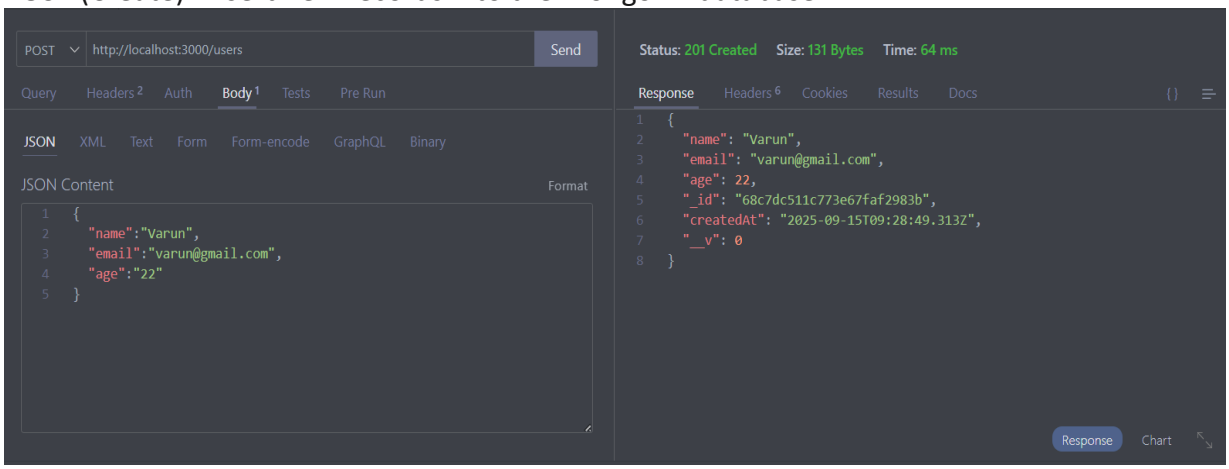
```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const User = require('./models/user');

const app = express();
const PORT = 3000;

// Middleware
app.use(bodyParser.json());
```

```
// Connect to MongoDB
 mongoose.connect('mongodb://127.0.0.1:27017/exp3')

.then(() => console.log('Connected to MongoDB'))
.catch((err) => console.error('MongoDB connection error:', err));

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

// a. Create a New User (POST)
app.post('/users', async (req, res) => {
  try {
    const { name, email, age } = req.body;
    const newUser = new User({ name, email, age });
    const savedUser = await newUser.save();
    res.status(201).json(savedUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
// b. Retrieve All Users (GET)
app.get('/users', async (req, res) => {
  try {
    const users = await User.find().lean();
    res.json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// c. Retrieve a Single User by ID (GET)
app.get('/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id).lean();
    if (user) {
      res.json(user);
    } else {
      res.status(404).json({ message: 'User not found' });
    }
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// d. Update a User by ID (PUT)

app.put('/users/:id', async (req, res) => {
```

```
  try {
    const { name, email, age } = req.body;
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      { name, email, age },
      { new: true, runValidators: true }
    );
    if (updatedUser) {
      res.json(updatedUser);
    } else {
      res.status(404).json({ message: 'User not found' });
    }
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
// e. Delete a User by ID (DELETE)
app.delete('/users/:id', async (req, res) => {
  try {
    const deletedUser = await User.findByIdAndDelete(req.params.id);
    if (deletedUser) {
      res.json({ message: 'User deleted successfully' });
    } else {
      res.status(404).json({ message: 'User not found' });
    }
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

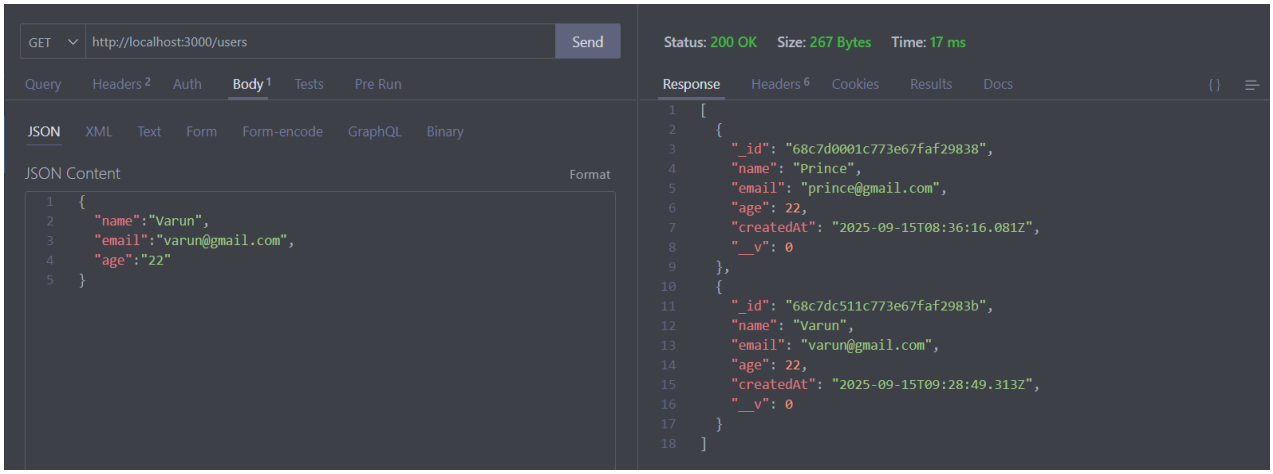
Output:

- POST (Create): Insert new records into the MongoDB database.

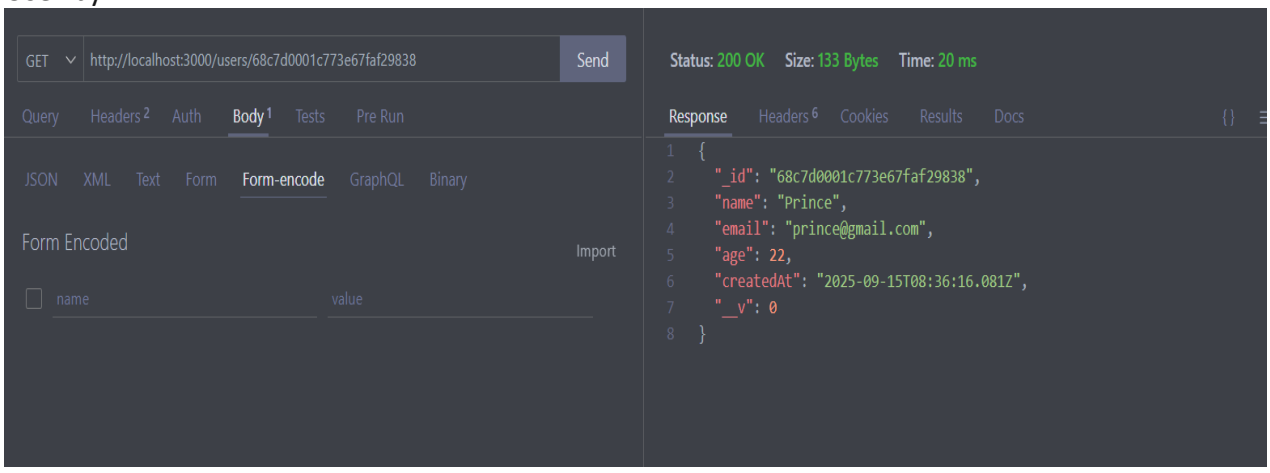


- GET (Read): Retrieve data from the database.

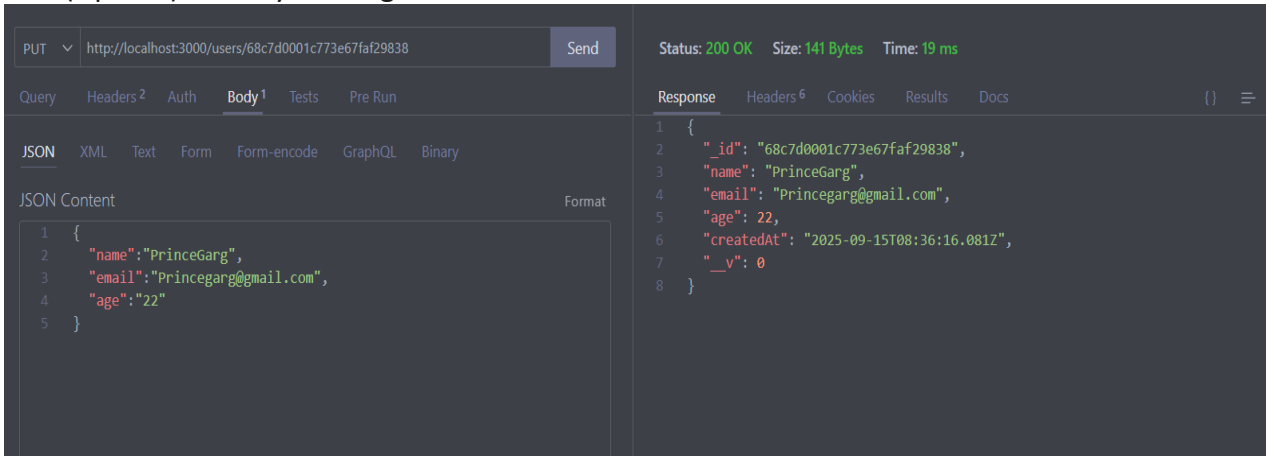
  All users:



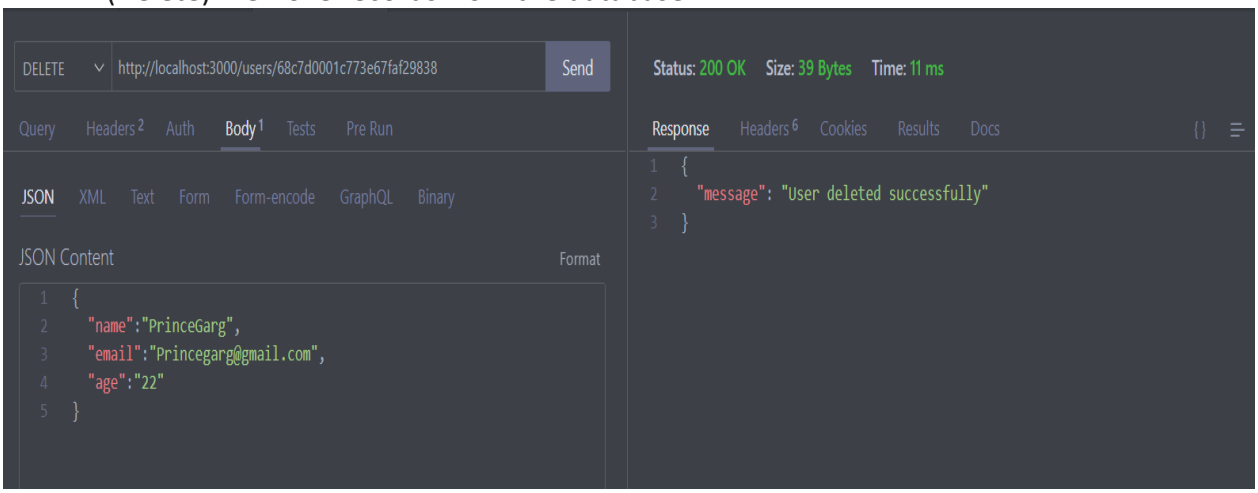
  User by ID:



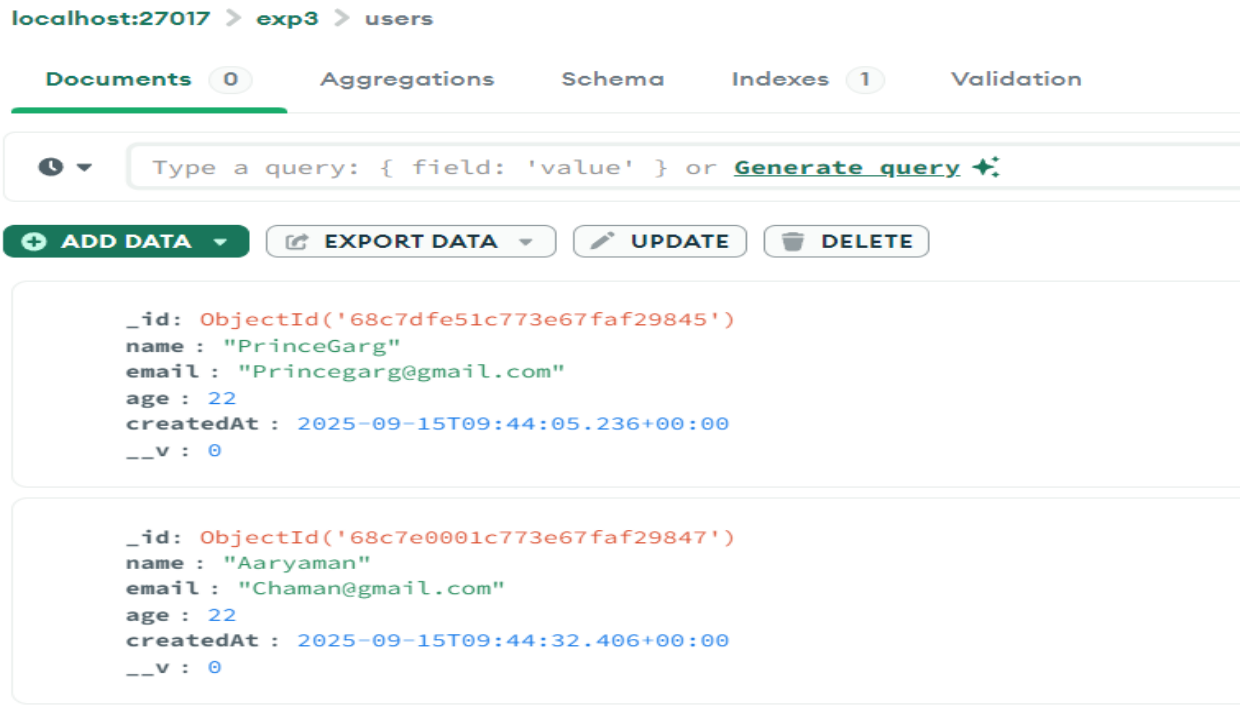
- PUT (Update): Modify existing records in the database.



- DELETE (Delete): Remove records from the database.



- MongoDB output:



Learning Outcomes:

a. Establish a connection between a Node.js application and MongoDB using Mongoose.

b. Perform CRUD operations effectively using Mongoose methods.

c. Gain practical experience in building database-driven applications.